

Week 10: Dynamic Properties and Queues (FIFO)

CMPT 211: Introduction to Software Development

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

Upgrading the Blueprint: Dynamic Properties

Last week, we hardcoded our student's GPA to zero: `self._gpa = 0.0`.

The Flaw: In professional software engineering, properties that represent measurements or averages should rarely be hardcoded static variables. They should be **dynamically calculated** from raw data to prevent the state from becoming outdated or corrupted.

Let's replace the static GPA with a list that holds raw course grades, and a method to populate it.

```
class ConcordiaStudent:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        # STEP 1: We initialize an empty list to act as the raw data ledger.
        self.grades = []

        # STEP 2: We create a Setter method to securely add new data.
    def add_grade(self, grade_percentage):
        # We validate the data BEFORE it touches our object's state!
        if 0 <= grade_percentage <= 100:
            self.grades.append(grade_percentage)
        else:
```

Step-by-Step: The Dynamic GPA Calculator

Now, instead of a variable, the GPA becomes a **Behavior** (a method) that computes the current truth every single time it is called.

```
class ConcordiaStudent:
    # ... (__init__ and add_grade methods from previous slide) ...

    def calculate_gpa(self):
        # STEP 1: Always protect against division by zero!
        # If the list is empty, return 0.0 immediately.
        if len(self.grades) == 0:
            return 0.0

        # STEP 2: sum() adds all numbers in the list. len() counts them.
        average_percentage = sum(self.grades) / len(self.grades)

        # STEP 3: Convert the 100-point scale to a 4.0 scale.
        gpa = (average_percentage / 100) * 4.0

        # STEP 4: Return the final output, rounded to 2 decimal places.
        return round(gpa, 2)
```

Inheritance in Action: The IT Student

Because we built this logic in the Parent class (ConcordiaStudent), our Subclasses inherit this dynamic capability instantly, with zero duplicated code!

```
# STEP 1: Instantiate the Child class (which calls super().__init__)
alice = ITStudent("Alice", 11111, "Admin")

# STEP 2: Use the inherited add_grade() method
alice.add_grade(85)
alice.add_grade(92)
alice.add_grade(78)

# STEP 3: Call the inherited calculate_gpa() method
# Python goes up the inheritance chain, finds the math logic, runs it
# on Alice's specific list of grades, and returns the result.
current_gpa = alice.calculate_gpa()

print(f"{alice.name}'s dynamic GPA is: {current_gpa}")
# Output: Alice's dynamic GPA is: 3.40
```

The Problem with Stacks (LIFO)

Now that Alice, Bob, and Charlie have their GPAs, they need to visit the Registrar's Office to print transcripts.

Why a Stack Fails Here:

- Last week, we used a **Stack (Last-In, First-Out)**.
- If Alice arrives at 9:00 AM, Bob at 9:15 AM, and Charlie at 9:30 AM...
- A Stack would force the clerk to serve Charlie first, because he is on "top" of the pile. Alice would be trapped at the bottom forever.
- In queuing systems (like networking packets, printer tasks, or humans in a line), this causes **Starvation**.

The Solution: The Queue (FIFO)

A Queue strictly enforces fairness. The first object to enter the structure is mathematically guaranteed to be the first object removed.

Alternative Visualization: The Queue as a Push-Down Tree

We normally visualize a Queue as a horizontal line. However, to understand the data movement, let's conceptualize it as a vertical, single-branching **Tree**.

1. The Enqueue (Inserting the Root)

- When a new student arrives, they do not go to the bottom. They are injected at the very top as the **New Root** of the tree.
- The former root (and all nodes attached below it) are physically pushed *down* the branches to make room.

2. The Dequeue (Harvesting the Leaves)

- Because every new arrival pushes the older data further and further down the tree, the *oldest* data ends up at the very bottom (the "Leaf").
- When the system is ready to process an item, it **harvests the leaf**.
- *The Root is the newest data; the Leaf is the oldest data.*

Implementing the Queue in Python

To build this "Push-Down Tree" (FIFO Queue) in Python, we encapsulate a list and strictly control the indices we insert and pop from.

```
class StudentQueue:
    def __init__(self):
        # The private list acting as our tree trunk
        self._lineup = []

    # BEHAVIOR 1: Enqueue (Insert new Root)
    def enqueue(self, student):
        # Step 1: We use list.insert(0, item) to force the new student
        # into the 0th index (the Root).
        # Step 2: Python automatically pushes all older items down one index.
        self._lineup.insert(0, student)
        print(f"{student.name} became the new root of the queue.")

    def size(self):
        return len(self._lineup)
```

Implementing the Dequeue Method

If the newest items are at Index 0 (the Root), the oldest items are pushed to the very end of the list (the Leaves). We must harvest from the end!

```
class StudentQueue:
    # ... (__init__ and enqueue methods here) ...

    # BEHAVIOR 2: Dequeue (Harvest the oldest Leaf)
    def dequeue(self):
        if self.size() == 0:
            raise IndexError("Cannot dequeue from an empty line!")

        # Step 1: .pop() WITHOUT an index argument automatically
        # removes and returns the VERY LAST item in the list.
        # Step 2: This guarantees First-In, First-Out (FIFO).
        served_student = self._lineup.pop()
        return served_student
```


Execution: Serving the Students

Let's watch the Push-Down Tree in action.

```
alice = ITStudent("Alice", 11111, "Admin")
bob = CSSStudent("Bob", 22222, "bob_dev")

# 1. Open the office line
registrar = StudentQueue()

# 2. Enqueue Phase
registrar.enqueue(alice) # Alice is Root (Index 0)
registrar.enqueue(bob)   # Bob becomes Root (Index 0), Alice pushed to Leaf (Index 1)

# 3. Dequeue Phase (Harvesting the Leaf at the end of the list)
next_student = registrar.dequeue()

print(f"Now serving: {next_student.name}")
# Output: Now serving: Alice (First-In, First-Out!)
```

Lab 10 Preparation: The Sliding Window Averager

In Lab 10, you will combine the Dynamic Averaging we learned today with the Queue data structure to build a **Sliding Window Averager**.

The Concept: Imagine we only want to calculate the GPA of the **last 3 assignments**.

- We create a Queue with a strict maximum size of 3.
- As assignment grades are **enqueued** (inserted as new roots), the older grades are pushed down.
- When the 4th grade arrives, the tree is full! To make room, we **dequeue** (harvest and discard) the oldest leaf.
- This creates a "window" of exactly 3 items that constantly slides forward through time, tracking only the most recent data.

Questions?

Review Miller & Ranum Chapter 3 for more on Queues before Lab 10!